Experiment 1: Matrix Operations in C

Aim:

To write a C program to perform basic matrix operations such as addition, subtraction, multiplication, and transpose of a matrix using two-dimensional arrays.

Objectives:

- To understand the concept of two-dimensional arrays in C.
- To perform arithmetic operations on matrices.
- To implement matrix addition and subtraction.
- To implement matrix multiplication using nested loops.
- To find the transpose of a matrix
- To develop programming skills using arrays and loops.

Theory:

A matrix is a rectangular arrangements of elements organized in rows and columns. In C programming, matrices are represented using two-dimensional arrays. Matrix operations are widely used in computer graphics, image processing, machine learning, scientific computing, and engineering applications.

1. **Matrix Addition**

Condition: Both matrices must have the same number of rows & columns.

Formula: `C[i][j] = A[i][j] + B[i][j]`

2. **Matrix Subtraction**

Condition: Both matrices must have the same dimensions.

Formula: $C[i][j] = A[i][j] - B[i][j]$

3. Matrix Multiplication

Condition: Number of columns of the first matrix must be equa to the number of rows of the second matrix.

Formula: $C[i][j] = \Sigma (A[i][k] * B[k][j])$; $k=0$ to $n-1$

4. Transpose of a Matrix

The transpose of a matrix is obtained by interchanging its rows and columns.

Formula: $A^T[i][j] = A[j][i]$

These operations are implemented using two-dimensional arrays, nested loops, and conditional statements.

Algorithm/Pseudo Code:

```
Algorithm Addition (A matrix, B matrix, rows, cols):
    Create matrix Result of size rows * cols
    For i from 0 to rows - 1:
        For j from 0 to cols - 1:
            Result[i][j] = A[i][j] + B[i][j]
    Return Result.

Algorithm Subtraction (Matrix A, Matrix B, rows(m), cols(n)):
        Create matrix Result of size m*n
        For i from 0 to m-1:
            For j from 0 to n-1:
```

Sundaram
FOR EDUCATIONAL USE

```
        Result[i][j] = A[i][j] - B[i][j]
    Return Result.

Algorithm Multiplication (Matrix A, Matrix B, rowsA, colsA, colsB):
// rowsA is m, colsA is n, colsB is p
Create matrix Result of size m * p

For i from 0 to m-1:
    For j from 0 to p-1:
        Result[i][j] = 0        //Initialize all elements to 0

For i from 0 to m-1:
    For j from 0 to p-1:
        For k from 0 to n-1:
            Result[i][j] += A[i][k] * B[k][j]
Return Result

Algorithm Transpose (Matrix A, m, n):
    Create matrix Transpose of size m*n
    For i from 0 to m-1:
        For j from 0 to n-1:
            Transpose[j][i] = A[i][j]
    Return Transpose.
```

FAQs:

1. Explain matrix addition, subtraction, multiplication & transpose with suitable examples and state the conditions required for each operation.

Ans. i. Matrix Addition:

Condition: Both matrices must have same number of rows & columns.

Formula: $C[i][j] = A[i][j] + B[i][j]$

Example: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ $B = \begin{bmatrix} 5 & 7 \\ 6 & 8 \end{bmatrix}$ $C = \begin{bmatrix} 6 & 9 \\ 9 & 12 \end{bmatrix}$

ii. Matrix Subtraction:

Condition: Both matrices must have the same dimensions.

Formula: $C[i][j] = A[i][j] - B[i][j]$

Example: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$ $C = \begin{bmatrix} -4 & -4 \\ -4 & -4 \end{bmatrix}$

iii. Matrix Multiplication:

Condition: Number of columns of the first matrix must be equal to the number of rows of second matrix

Formula: $C[i][j] = \Sigma (A[i][k] * B[k][j])$; $k = 0$ to $n-1$

Example: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$ $C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

iv. Transpose of Matrix:

Condition: The transpose of a matrix is obtained by interchanging its rows and columns

Formula: $A^T[i][j] = A[j][i]$

Example: $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ $A^T = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$

Sundaram
FOR EDUCATIONAL USE

2. Describe how two-dimensional arrays are used to represent matrices in C prog. Explain the process of accessing and manipulating matrix elements using nested loops.

Ans. A 2D array is essentially an "array of arrays". It is declared by specifying the datatype, followed by array name and size.

Eg. int A[10][10];

The data of the elements is stored row wise in the computer memory.

Accessing elements using ~~loops~~indices:

The indexing in C starts from 0 i.e. 1st row=[0], 1st col=[0]

To thus access any element of the matrix, we can specify their index values. (matrix[row_index][col_index])

Eg. If we want to access 3rd element of 1st row, then we can use A[0][1]

Manipulating matrix elements using nested loops:

The number of nested loops depends upon the number of rows and columns of the matrix. Here, it's 2,2 so, these will be 2 ~~nested~~ loops.

a. Outer loop: tracks the row index (i)

b. Inner loop: tracks the column index (j) within the specific row.

For every single step the outer loop takes, the inner loop runs completely from start to finish.

3. Write the algorithm for performing matrix multiplication and explain why compatibility between matrices is necessary before multiplication.

Ans. Algorithm Matrix Multiplication()

Step 1: Start

Step 2: Read dimensions R1, C1 for A and R2, C2 for B.

Step 3: Check compatibility.

If C1 is Not equal to R2:

Print "Matrices are incompatible for multiplication!"

Go to step11

Step 4: Read elements of Matrix A (size R1 x C1).

Step 5: Read elements of Matrix B (size R2 x C2).

Step 6: Create a Result Matrix C of size R1 x C2.

Step 7: Loop through each row of Matrix A:

For i from 0 to R1-1:

Step 8: Loop through each column of Matrix B:

For j from 0 to C2-1:

Step 9: Initialize the element:

C[i][j] = 0

Loop to calculate the dot product:

For k from 0 to C1-1:

C[i][j] += A[i][k] * B[k][j]

Step 10: Print Matrix C.

Step 11: Stop.

Matrix multiplication relies on the dot product, where you multiply pairs of elements from a row of the first matrix by a column of the second matrix and sum them up. Because of this pairing, the

Sundaram®

FOR EDUCATIONAL USE

first matrix must have exact same number of columns as the second matrix has rows. Without this compatibility, a row from the first matrix and a column from the second would have different lengths, leaving elements without a pair and making the calculation mathematically impossible.

4. Discuss the applications of matrix operations in real-world domains such as computer graphics, image processing, machine learning, engineering, and scientific computing.

Ans. Matrix can store and process data in grids and also allow computers to run thousands of calculations simultaneously. Their applications in real-world app-domains are as follows:

i. Computer graphics:

Every 3-D video game or animation relies entirely on matrix to render shapes on screen.

a. Vector transformations: Moving, rotating, or scaling a 3D object is done by multiplying its ~~vector~~ co-ordinates by specific transformation matrices.

b. Projection: Matrices calculate how a 3D environment should flatten onto 2D monitor so that objects naturally look smaller when they are farther away.

ii. Image Processing:

a. Filters and Blurring: Image filters use matrix operations called convolutions, sliding a tiny matrix over pixel values to sharpen edges or blur details.

b. Compression: Formats like JPEG use matrix transformations to isolate and remove redundant visual details, drastically shrinking file sizes without ruining file quality.

iii. Machine learning & AI

a. Neural Network Layers: AI models pass data through networks by multiplying input data matrices with large matrices of learned weights.

b. GPU acceleration: Graphics Processing Units are explicitly engineered to run these massive, repetitive matrix operations simultaneously at lightning speed.

iv. Engineering:

a. Structural analysis: Engineers convert structural forces into massive matrix equations to simulate how bridges or aircrafts handle stress.

b. Electrical circuits: Interconnected power circuits grids are molded as matrix systems to easily solve for unknown voltages and currents accross thousands of nodes.

v. Scientific computing:

a. Weather forecasting: Meteorologists use grids of matrices to solve complex fluid dynamics equations that track global atmospheric changes.

5. Explain the role of nested loops in implementing matrix operations in C. Illustrate how nested loops are used for addition, multiplication, and transpose of matrices.

Ans. Nested loops are essential for matrix operations because they create a co-ordinate system ([i][j]) to traverse rows and columns. The outer loop selects the row index (i), and the inner loop sweeps across every column index (j) within that row.

They are used as follows for the listed operations:

i. Matrix Addition (2 loops)

Scans both matrices at the same time pace to add corresponding elements.

```
for (int i=0; i<rows; i++){
    for (int j=0; j<cols; j++){
        res[i][j] = A[i][j] + B[i][j];
    }
}
```

ii. Matrix Multiplication (3 loops)

Requires a third loop (k) to calculate the dot product by travelling across row i of A and down column j of B simultaneously

```
for (int i=0; i<rowsA; i++){
    for (int j=0; j<colsB; j++){
        res[i][j] = 0;
        for (int k=0; k<colsA; k++){
            res[i][j] += matA[i][k] * matB[k][j];
        }
    }
}
```

iii Transpose of Matrix (2 loops)

Flips the grid structuse by writing element at [i][j] to [j][i]

```
for (int i=0; i<rows; i++){
    for (int j=0; j<cols; j++){
        transpose[j][i] = matrix[i][j];
    }
}
```

Pseudo code (contd...)

```
Function main ():
    Display menu: "1. Addition, 2. Subtract, 3. Multiply, 4. Transpose"
    Get user choice and dimension(s) of matrices depending on choice.
  Switch (ch):
  Case 1: If (rowA != rowB) or (colA != colB):
            Print "Error: Dimension must match!"
            Break
          Result = call matrix Addition (A, B, rowA, colA)
            Break
  Case 2: If (rowA != rowB) or (colA != colB):
            Print "Error: Dimension must match!"
            Break
          Result = call matrix Subtraction (A, B, rowA, colA)
          Break.
  Case 3: If (rowA != rowB) or (colA != colB):
            Print "Error: Dimension must match!"
            Break
          Result = call multiplication (A, B, rowA, colA, colB)
          Break
  Case 4: Result = call transpose (A, rowA, colA)
            Break
  Default: Print "Invalid choice!"
            Break.
```

Sundaram®
FOR EDUCATIONAL USE